EECE 490 HACKATHON · STORIES COFFEE

Stories Pair

Turning half-orders into profitable pairs,
one till suggestion at a time.

72%

of Stories orders are half a pair

Project report: data, models, decision engine, learning engine, value and pilot plan

Data: 51,177 product lines · 33,887 orders · Harbour, Garden, Uptown · Jan–Jun 2026

Team: ‹Member 1 full name› · ‹Member 2 full name›

Repository: github.com/mouhammed-soumaille/EECE-490-Hackathon

SUMMARY

The opportunity in one page

72% of Stories' orders contain only a drink or only food. Stories Pair is a till engine that, when the first item is rung, suggests the item from the other side of the pair with the highest *expected extra gross profit* at that branch, net of customers who would have bought it anyway. It is built only on what the data supports, starts as a 135-row lookup table any POS can use, and then learns from every accept and decline.

72%	**$17k**	**+27%**	**$0**
of orders are half a pair (measured)	extra gross profit a year, base case (estimate)	more with the learning engine (simulation)	discount given away at base assumptions

- **What the data supports.** Classic market-basket analysis finds no significant product pairs. Among orders that do pair, the *category* of the add-on is strongly predictable from the first item: a matcha or tea rung first is followed by bakery 78% of the time; a sandwich or meal by a cold drink 41% of the time (cold drinks are 18% of drink lines).
- **What it does not support.** Product-level pairs, personal preferences behind loyalty IDs, and context-heavy machine learning all fail held-out tests. Discounts did not change baskets.
- **Value.** At base assumptions Stories Pair adds about $16,811 of gross profit a year (P10–P90 $11,835–$26,878), 15% more than suggesting the best-seller and 37% more than a classic 10% combo, which would also hand out $3,858 a year in discounts.
- **Learning.** The v2 engine (Thompson-sampling bandit) earns about $27,379 vs $21,478 for v1 in its first year across 30 simulated customer truths, and beats the best-seller rule and classic combo in all of them.
- **Next step.** Two weeks of accept/decline logging, then a six-week odd/even receipt A/B test, measure the one number history cannot: how often customers accept.

Most orders stop at one item

The file has one row per product line: 51,177 lines forming 33,887 orders at three branches between 1 January and 30 June 2026. Before analysis we ran a data contract: line revenue and gross profit reconcile exactly (0 mismatches), each Transaction ID belongs to one branch, timestamp, customer and order type, and every product has one price and one cost per branch. 132 repeated lines (0.26%) were kept; removing them does not change the order mix.



Figure 1. Order composition. 40.9% drink-only, 30.9% food-only, 28.2% both.

What counts as an order

We tested merging receipts that share a branch and the exact second (a group paying separately). There are 30 such collisions, close to the ~21 expected by pure chance, and the same customer appears twice at a branch on the same day in under 1% of customer-days. We keep the Transaction ID as the order. We treat the first line of an order as the item rung first; the export does not record ring order, but section 2 shows the add-on depends on the first line more strongly than the pooled view, which is what an anchor item would produce.

From the till's point of view, when a drink is rung first only 25.6% of orders go on to include food; when a food is rung first, 31.4% go on to include a drink. The remaining 69–74% are the audience for a suggestion.

2 · WHAT THE DATA SUPPORTS

Pairs live at the category level

Whether an order pairs barely moves with branch, hour, weekday, order type or loyalty (Cramér's V ≤ 0.03); it moves with what is rung first (V 0.08–0.18). Rates are flat month to month (25–27% drink-first).

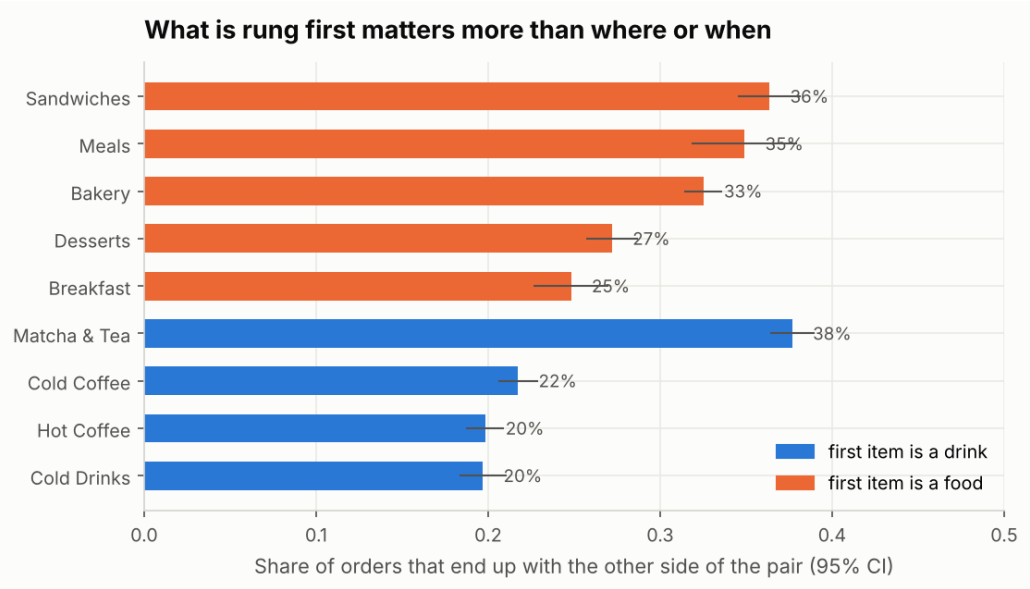


Figure 2. Share of orders that end up paired, by the first item's category (95% CI).

Which category is added depends strongly on the first item. Daypart, branch and order type are statistically significant but weak.

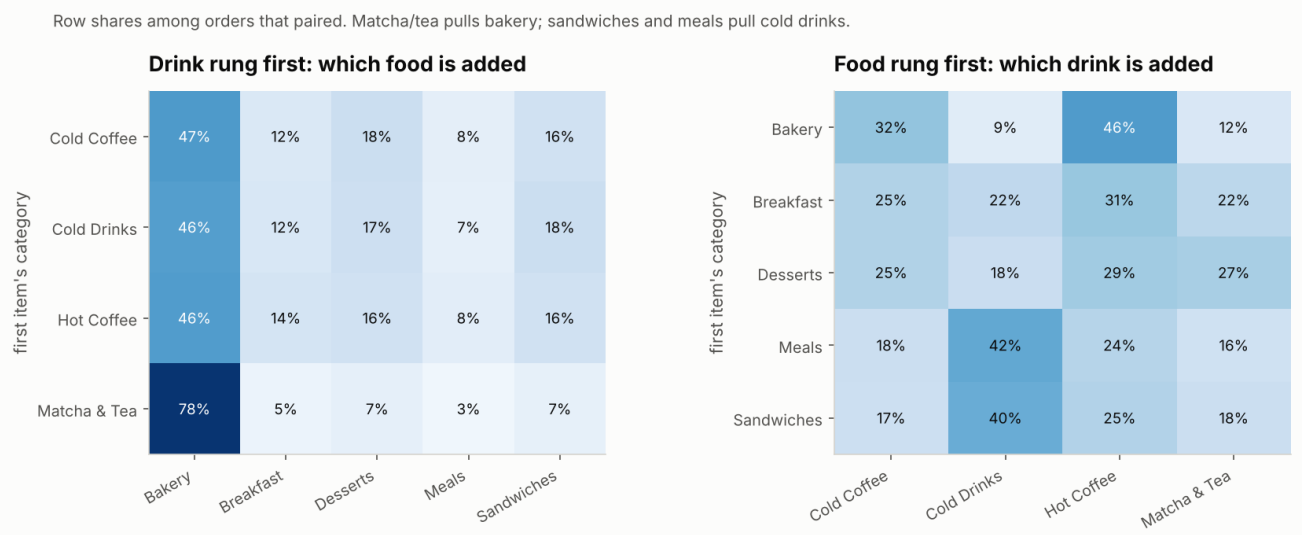


Figure 3. Among orders that paired: the category added, by the first item's category, in each direction.

Below the category, nothing

Across all orders no drink × food product pair is significant after a Benjamini–Hochberg correction (0 of 506), and even every category lift is at or below 1, because most orders are single-sided. Restricting to orders that pair reveals strong category pairs (cold drinks with sandwiches, lift 1.8; with meals, 1.7; matcha/tea with bakery, 1.2; all $q < 0.001$). Some product pairs look significant there too, but every one sits inside a significant category pair, and none of 20 within-category tests survives a Bonferroni correction: once the categories are known, the specific drink says nothing about the specific food.

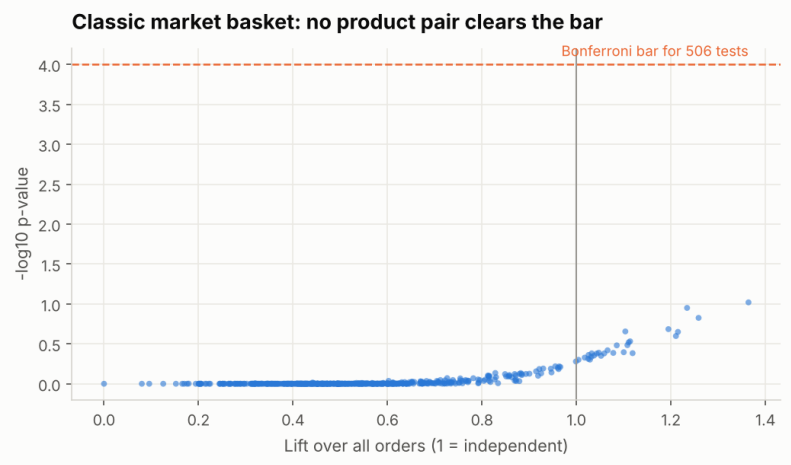


Figure 4. Classic market basket over all orders: no product pair clears the multiple-testing bar.

Loyalty IDs carry no personal preference

3,494 IDs appear a median of 6 times; 94% appear at more than one branch. A returning customer repeats their previous food choice 5.0% of the time against 5.2% expected by chance (z = −0.85). Personalisation would add privacy exposure and complexity for no measurable gain, so we rejected it.

Held-out benchmark: which method predicts the add-on?

Predictor (trained Jan–May, tested on June)	hit@1	hit@3	log loss
Global popularity	0.112	0.277	2.937
Branch x daypart popularity	0.109	0.263	2.971
Product association rules	0.113	0.268	2.935
Category-aware (Stories Pair)	0.110	0.271	2.885

Product association rules are no better than plain popularity on unseen orders. The category-aware structure gives the best probabilities (lowest log loss), which is what the engine multiplies by profit.

3 · MODELS VS BASELINES

Simple wins when it ties

The engine needs two measured probabilities when the first item is rung: **Q**, the chance the order pairs anyway, and the chance each category is added if it does. Protocol: train January–April, choose on May, refit January–May, report June, which no decision touched.

Model for Q (will the order pair?)	May AUC	June AUC	June log loss
Base rate (always predict the average)	0.500	0.500	0.5953
Segment rate: first-item category	0.601	0.597	0.5832
Segment rate: first product	0.599	0.595	0.5839
Logistic regression (all context)	0.602	0.593	0.5843
Gradient boosting (all context)	0.600	0.587	0.5855

A paired bootstrap on June shows logistic regression is no better than the one-line baseline (AUC difference −0.004, 95% CI −0.009 to +0.001) and gradient boosting is worse (−0.010, CI −0.017 to −0.005). **The engine uses the baseline**: same accuracy, nothing to maintain.

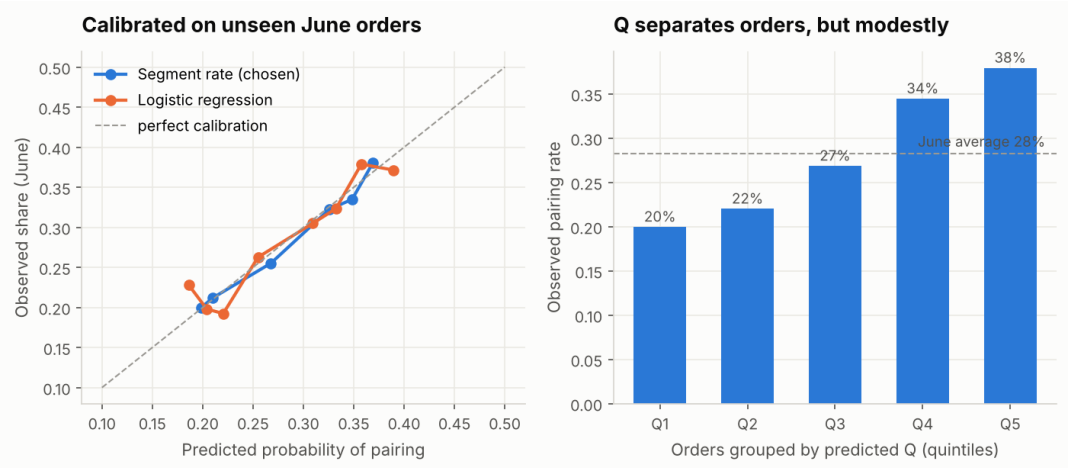


Figure 5. The chosen model is calibrated on unseen June orders and separates them modestly.

Category model, drink rung first	May log loss	June log loss	vs global
Global category shares	1.2002	1.1861	+0.0%
Shares by daypart	1.2031	1.1835	+0.2%
Shares by first-item category	1.1487	1.1315	+4.6%
Shares by first-item category x daypart	1.1593	1.1346	+4.3%
Multinomial logistic regression	1.1536	1.1287	+4.8%

Selection rule fixed before looking at June: best May log loss, simplest model within 0.5%. "Shares by first-item category" wins in both directions. Daypart is statistically significant ($p < 1e\text{-}30$) yet does not improve prediction on unseen months: significance is not usefulness.

4 · THE DECISION ENGINE

Score profit, not popularity

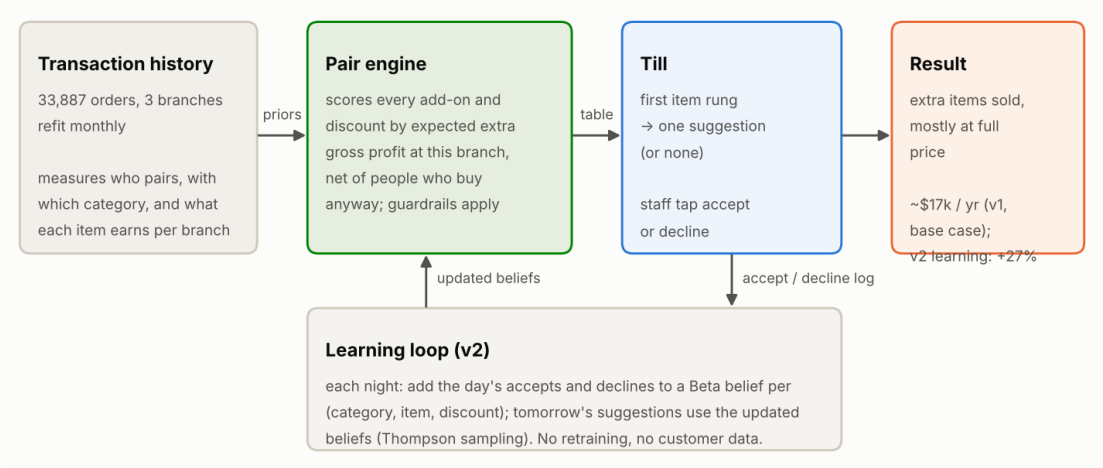


Figure 6. How Stories Pair works end to end.

For each candidate add-on *i* and discount *d* (0, 5, 10 or 15%), the engine computes:

$$\text{gain} = (1 - Q) \cdot a_i(d) \cdot gp_i(d) \quad \text{new sales from people who would not have added anything}$$
$$- Q \cdot s_i \cdot (p_i - p_i(d)) \quad \text{discount handed to people who would buy } i \text{ anyway}$$
$$+ w \cdot Q \cdot (1 - s_i) \cdot (gp_i(d) - gp_{other}) \quad \text{people who switch to the suggestion}$$

Q, s_i, prices and costs are measured. Acceptance $a_i(d)$ is assumed in v1 (6% for an average-appeal item, proportional to how often the item is chosen organically, +10% per 10% off) and learned in v2. Guardrails: discount ≤ 15% (the deepest in Stories' data), ≥ 40% margin after discount, the add-on costs at most 2.5× the first item, no suggestion worth under $0.05, never repeat a declined offer.

What the engine decides

- **No discounts at base assumptions.** Discounting pays only if 10% off raises acceptance by about a quarter or more. The data shows none: when the first drink was discounted, pairing was 25.1% vs 25.7% at full price (p = 0.65). The naive comparison (34% vs 25%) is a counting artifact of bigger orders.
- **Branch-aware.** Garden pays 1.43× Harbour's unit cost on all 45 products. At Garden, iced coffees and cold drinks are paired with an Egg Sandwich; at Harbour and Uptown with a Chicken Pesto Sandwich, which earns $3.36 a unit at Garden vs $6.12 at Harbour.
- **A 135-row playbook.** The chosen models depend only on branch and first item, so the whole v1 engine compiles to a lookup table (outputs/playbook.csv) a POS can load on day one.

First item (Harbour)	Suggest	Pair total	Pairs anyway	Expected gain
Almond Croissant	Iced Latte	$10.71	32%	$0.41
Brownie	Iced Matcha	$12.60	27%	$0.40
Fresh Orange Juice	Chicken Pesto Sandwich	$17.33	20%	$0.37
Granola Yoghurt	Iced Latte	$13.86	25%	$0.36
Cappuccino	Egg Sandwich	$12.81	20%	$0.35
Iced Mocha	Chicken Pesto Sandwich	$17.85	22%	$0.34
Chai Latte	Butter Croissant	$9.34	38%	$0.31
Chicken Salad Bowl	Peach Iced Tea	$17.74	35%	$0.29

Excerpt from the playbook: the best suggestion for one item of each category at Harbour.

5 · VALUE (ESTIMATES)

What it is worth, and what could change it

The engine was trained on January–May and replayed on all 6,906 June orders it never saw; every policy is scored with the same acceptance model, then scaled to a year using the six-month average volume (68,336 orders).

Policy	Extra gross profit / yr	Discount given / yr	Gain per suggestion
Stories Pair (item + discount optimised)	$16,811	$0	$0.31
Suggest the best-seller (full price)	$14,608	$0	$0.27
Classic combo: best-seller at 10% off	$12,249	$3,858	$0.22
Suggest the highest-profit item (full price)	$11,236	$0	$0.21

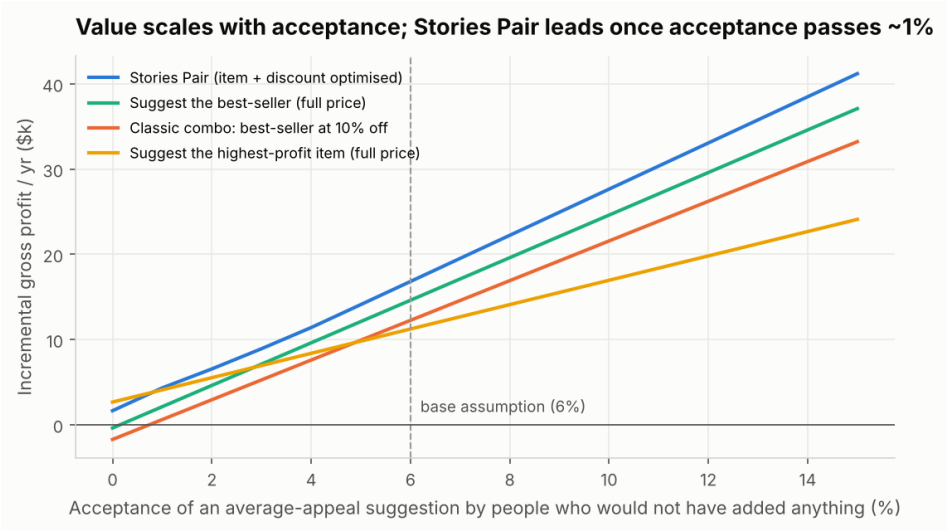


Figure 7. Value scales with acceptance; each percentage point is worth about $2,650 a year.

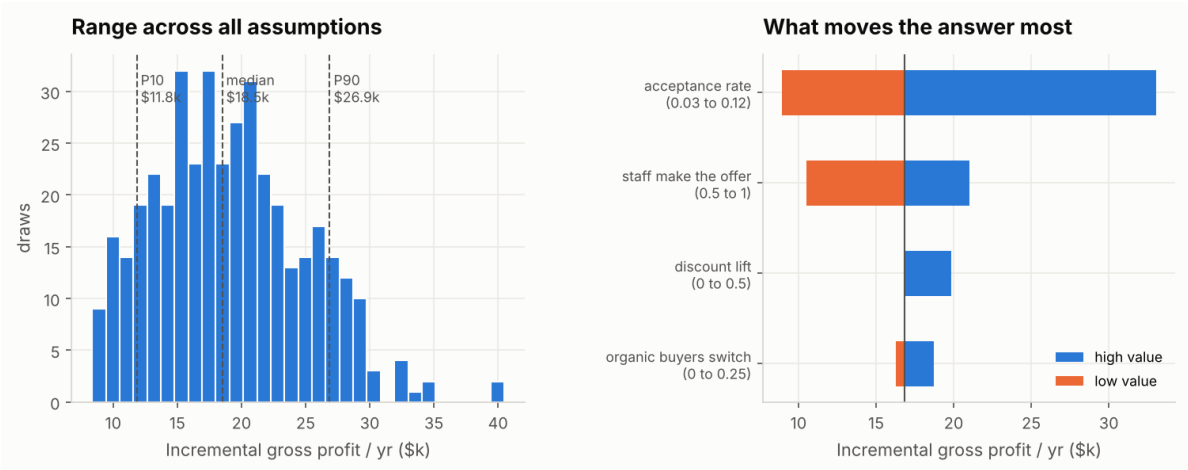


Figure 8. Varying every assumption at once: P10 $11,835, median $18,497, P90 $26,878; no draw loses money. Acceptance is the dominant driver.

6 · STORIES PAIR v2

An engine that learns at the till

v2 replaces the acceptance assumption with a Thompson-sampling bandit. Each arm (first-item category, suggested item, discount) keeps a Beta belief about its take rate, warm-started at v1's estimate with the weight of 30 suggestions. For each order the engine draws from every allowed arm's belief, values the draw with that branch's economics, and suggests the best draw; each night the day's accepts and declines update the beliefs. Acceptance is learned jointly across branches because behaviour does not differ by branch; profit always uses each branch's own prices and costs.

We tested it against 30 simulated truths drawn from the assumption ranges, each with item-level deviations of about ±60% from v1's "acceptance follows popularity" rule, over a year of orders resampled from real April–June orders, against an oracle that knows the truth.

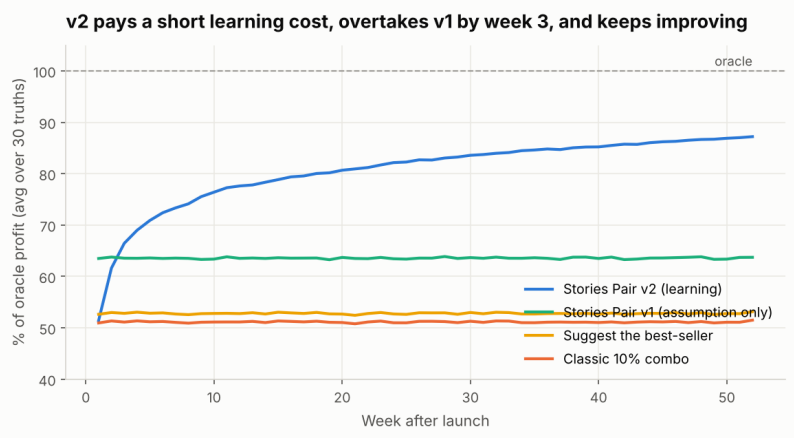


Figure 9. Share of the oracle's profit captured, averaged over 30 truths.

Policy (first year, 80% of orders shown a suggestion)	Mean	P10	P90	v2 wins in
Oracle (knows the truth)	$33,387	$22,789	$42,706	–
Stories Pair v2 (learning)	$27,379	$17,570	$38,524	–
Stories Pair v1 (assumption only)	$21,478	$15,028	$33,567	87% of truths
Suggest the best-seller	$17,938	$10,764	$27,495	100% of truths
Classic 10% combo	$17,425	$10,392	$26,355	100% of truths

v2 pays a short exploration cost (51% of the oracle in week 1 vs 64% for v1), overtakes v1 by week 3, and reaches 87% of the oracle by week 52; v1 never moves. Its discount use rises with the true discount effect (r = 0.72) without being told. These figures span the whole assumption range, so compare policies within this table rather than with section 5.

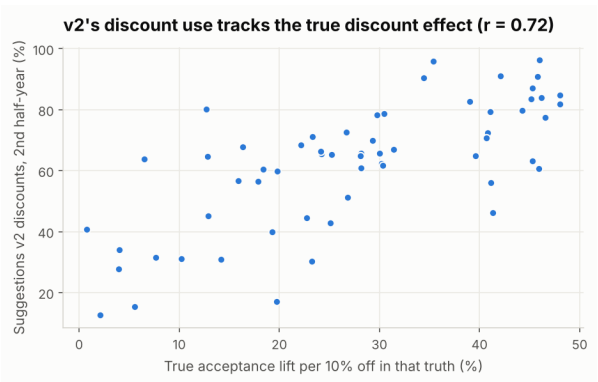


Figure 10. v2 learns how much discounting pays in each simulated truth.

7 · PRODUCT

From notebook to till

- **Till app** (app/index.html): a browser till with no server. Staff ring the first item, see the suggested pairing with its price and a one-line script, and tap accept or decline. A staff-only panel shows the options the engine scored, the chance the order pairs anyway, and the engine's live belief about the suggestion. A second tab runs a four-week simulated service of v2 against v1 and a classic combo. Its JavaScript engine reproduces the Python engine's decisions exactly.
- **Colab demo** (notebook 06): the same flow as a Gradio app with a public link for presentations.
- **Integration path.** Day one: load outputs/playbook.csv into the POS as a lookup. With the pilot: log accept/decline per suggestion. With v2: a nightly job updates two counts per arm and republishes the table; the till does one random draw per arm. No GPU, no retraining, no customer data.
- **Business-editable.** Assumptions, guardrails and category compatibility live in config/*.json, so Stories can change a rule without touching code.

Pilot plan and decision rules

- **Step 1 · Acceptance logging, 2 weeks, all branches.** About 2,200 suggestions estimate a ~6% acceptance rate to within ±1 point. If acceptance is below ~2%, stop.
- **Step 2 · Odd/even receipt A/B test, 6 weeks.** Even Transaction IDs get the suggestion, odd ones do not. Primary metric: gross profit per order; secondary: pairing rate, acceptance, discount spend, complaints.
- **Step 3 · Switch on v2** on the logged data; test a discounted variant on one branch only if step 2 succeeds.

Detectable gain per order	Orders per arm	Days, all branches	Days, Harbour only
$0.10	12,190	131	271
$0.15	5,418	58	121
$0.20	3,048	33	68
$0.25	1,951	21	44
$0.30	1,355	15	31

80% power, 5% significance, order-level gross-profit SD $2.79. The base case predicts $0.25 per order.

8 · RIGOUR

Measured, estimated, rejected

Statement	Status
72% of orders are single-sided	Measured
Pairing depends on the first item's category, not on branch, hour or order type	Measured (tested, held-out)
Which category is added depends on the first item's category	Measured (tested, held-out)
No product-level pairing beyond the category	Measured (multiple-testing corrected)
Loyalty IDs show no personal preference	Measured
Garden unit costs are 1.43× Harbour on all 45 products	Measured
A discounted first drink did not change pairing	Measured (observational)
The first line of an order is the item rung first	Assumed (supported by evidence)
Acceptance 6%, switching 10%, staff compliance 80%, +10% per 10% off	Assumed (pilot measures)
~$16,811/yr (P10–P90 $11,835–$26,878)	Estimated
v2 +27% over v1 in year one	Simulated

Approaches we rejected

Approach	Why not
Product-level market basket / association rules	No significant pairs; no better than popularity on June data
Customer-level personalisation	No personal preference behind loyalty IDs; privacy cost
Gradient boosting / context-heavy ML for Q	Ties or loses to a one-line baseline on unseen months
Blanket combo discounts	Pays customers who would buy anyway; no discount effect in the data
Merging same-second receipts	Collisions at chance level
"Breakfast only before noon" rule	56% of breakfast items sell after 3 pm

Limitations

- Acceptance is the dominant unknown; every value figure scales with it. The pilot exists to measure it.
- Ring order is inferred, not recorded. Discount evidence is observational, and a discount shown upfront on an add-on is a different intervention.
- Six months of data: no full-year seasonality check. Gross profit is not net profit (no labour, waste or integration cost).
- The bandit simulation shares our model's structure; real customers may show effects it does not cover, such as suggestion fatigue or staff tone.
- Garden's cost gap is unexplained (supplier terms, recipe, waste or allocation). Bringing it to Harbour's unit costs would be worth up to ~$32k a year on its own, an upper bound.

Reproducibility

Every number in this report comes from the notebooks in the repository (00–07), which run top to bottom in Google Colab and are committed with their outputs. Shared code lives in src/storiespair; this report is rebuilt with python reports/build_report.py.